

# An Approach for Static Analysis of Fluent APIs

26M30498 | Dennis Prüne

PRG Lab

# Contents

- Background
- Related Work
- My Proposal
- Current Progress & Summary

## Background

# What are Fluent APIs?

```
List<Integer> ys = new ArrayList();  
for (int x : xs) {  
    if (x >= 0) {  
        ys.add(2 * x);  
    }  
}
```

### Imperative Style

- For-loops, if-clauses, ...
- Nested

```
List<Integer> ys = xs  
    .stream()  
    .filter(x -> x >= 0)  
    .map(x -> 2 * x)  
    .collect();
```

### Fluent Style

- Method chaining
- Flat

## Background

# Fluent APIs 🤝 Embedded DSLs

```
SELECT name, birth_year  
FROM Students  
WHERE birth_year == 2005
```

naïve embedding

```
Query query = SQL.query(  
    "SELECT name, birth_year "  
    + "FROM Students "  
    + "WHERE birth_year == 2005");
```

fluent embedding

```
Query query = SQL  
    .select("name, birth_year")  
    .from("Students")  
    .where("birth_year == 2005");
```

# Background

## The Tradeoff

### Advantages

- Less syntax to remember
- Dynamic data support

```
SQL.select(field).from("Students")
```

can be set  
programmatically



### Disadvantages

- NO static safety guarantees
  - NO syntax checking
  - NO type checking
  - NO semantic analysis

```
SQL.select("name, birth_year")  
  .from("Students")  
  .from("Teachers")
```

illegal syntax  
causes runtime error



## Related Work

# Safe Fluent APIs<sup>[EriLex, H. Xu, 2010][Fling, Gil & Roth, 2019][...]</sup>

Goal: Disallow bad method chains at compile time

1. Encode allowed chains as a CFG

```
Query ::= SQL.select(String selections)  
                .from(String source)  
                Where
```

```
Where ::=  $\epsilon$  | .where(String clause)
```

2. Generate a fluent API where this syntax is encoded in the type system

## Related Work

# Limitations

- Only ensures partial syntactic correctness
- Verbose error messages

SQL.select("1 + 6").from("Students")

realistic error message



syntax error which cannot  
be detected by safe fluent API



```
Test.hs:5:8:
  No instance for (Show r0) arising from a use of 'print'
  The type variable 'r0' is ambiguous
  Note: there are several potential instances:
    instance Show a => Show (Maybe a) -- Defined in 'GHC.Show'
    instance Show Ordering -- Defined in 'GHC.Show'
    instance Show Integer -- Defined in 'GHC.Show'
    ...plus 30 others
  In the expression: print
-- omit 28 lines --
Test.hs:9:34:
  No instance for (ShapeTransition
    (Node5 (Node9 (Node6 (Node10 (Node11 (Node1 ()))))) s0)
  arising from a use of 'shape'
  In the second argument of '(|>)', namely 'shape "dotted"'
  In the first argument of '(|>)', namely
    'begin |> digraph "small_graph" |> node "A" |> shape "rectangle"
    |> node "B"
    |> node "C"
    |> shape "doublecircle"
    |> edge "A"
    |> to "B"
    |> shape "dotted"'
  In the first argument of '(|>)', namely
-- omit 9 lines --
Test.hs:10:12:
  No instance for (EdgeTransition s0 s1) arising from a use of 'edge'
  The type variables 's0', 's1' are ambiguous
-- omit the rest and three other errors --
```

Proposal

# Static Analysis via Backtranslation

1. Find  
embedded  
language code

2. Translate to  
original DSL

3. Analyze w/  
existing tools

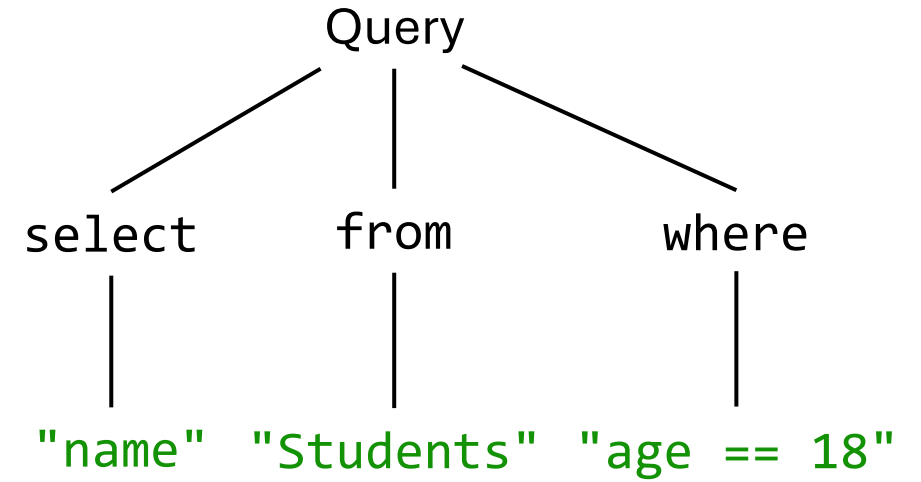
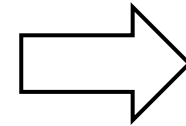
4. Project  
results onto  
input



# Proposal

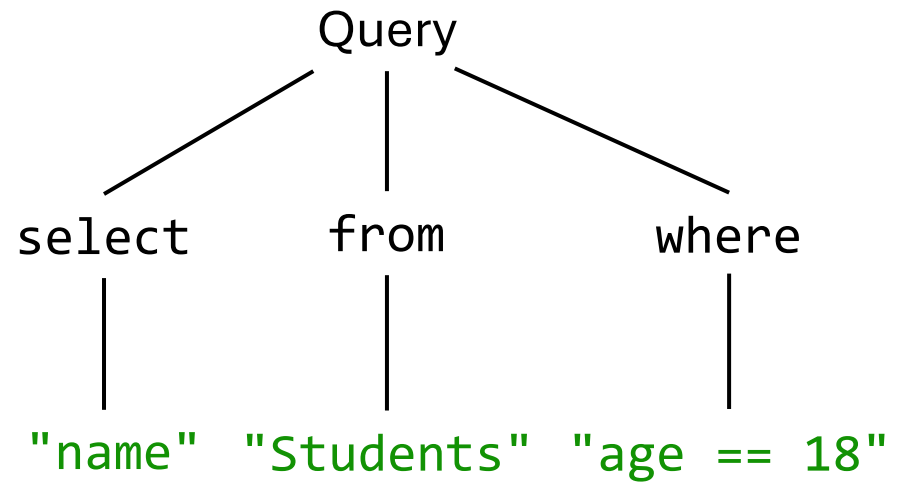
## 1. Find EDSL Code

```
class Bar {  
  ...  
  public void foo() {  
    ...  
    Query query = SQL.select("name")  
                    .from("Students")  
                    .where("age == 18");  
    ...  
  }  
}
```



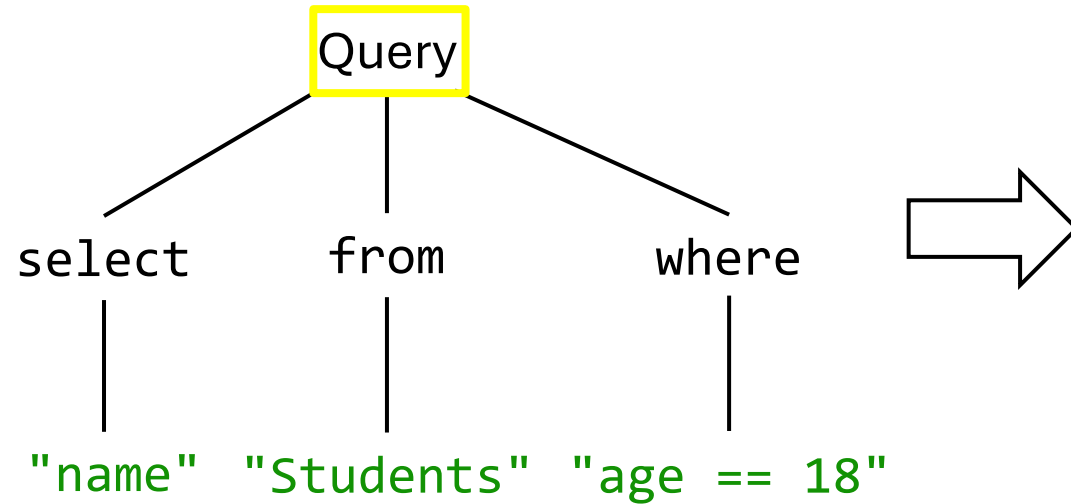
# Proposal

## 2. Backtranslation



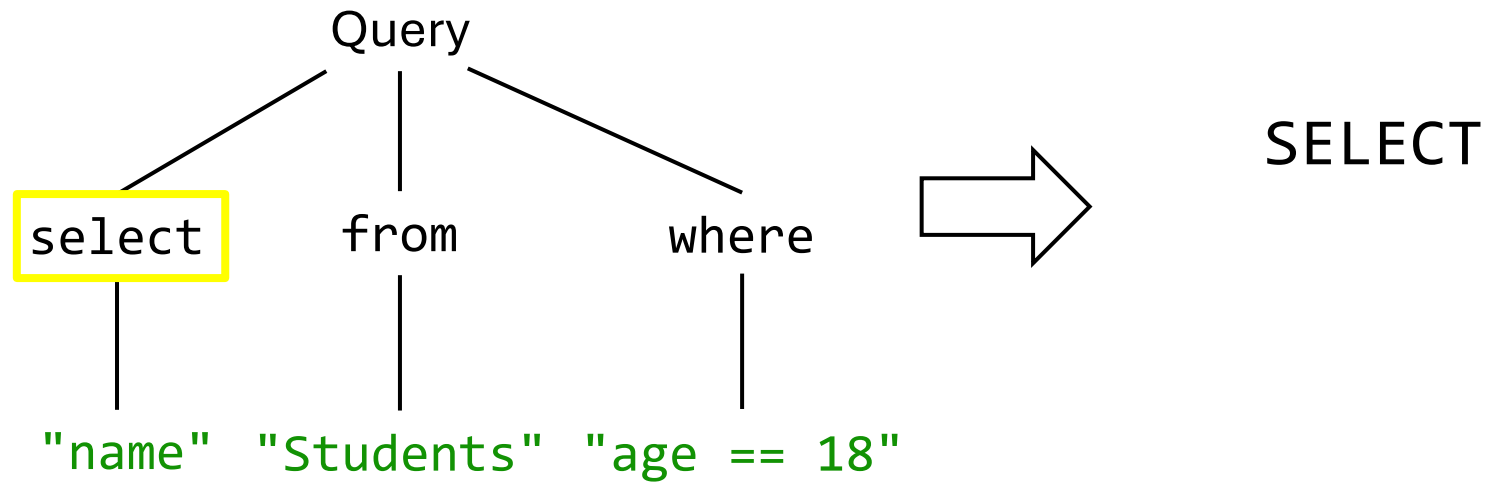
# Proposal

## 2. Backtranslation



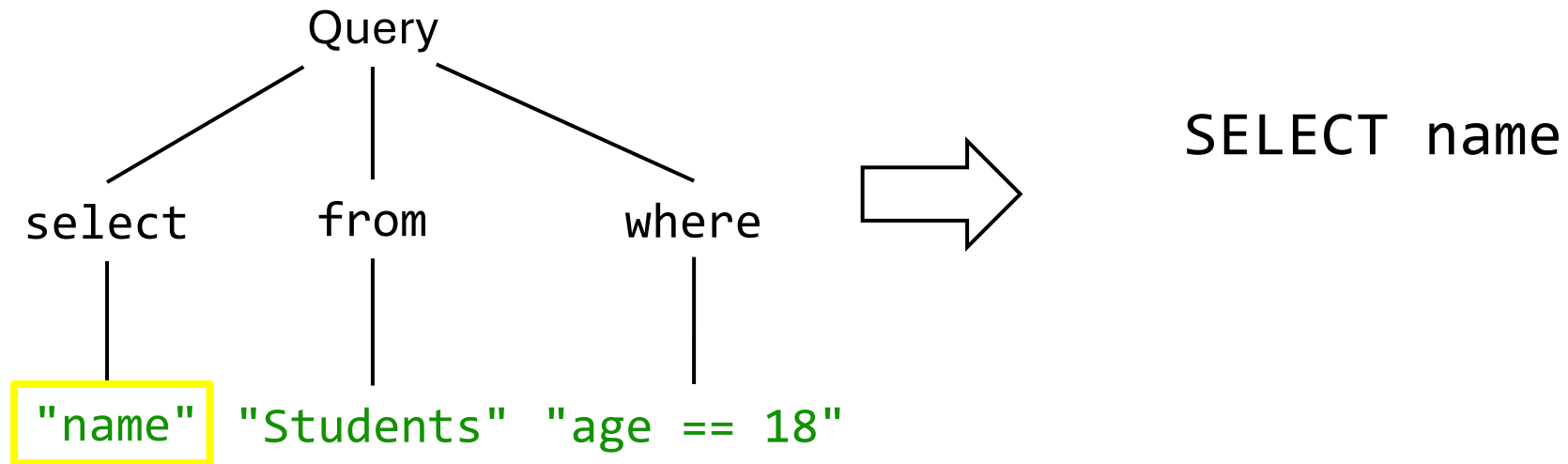
# Proposal

## 2. Backtranslation



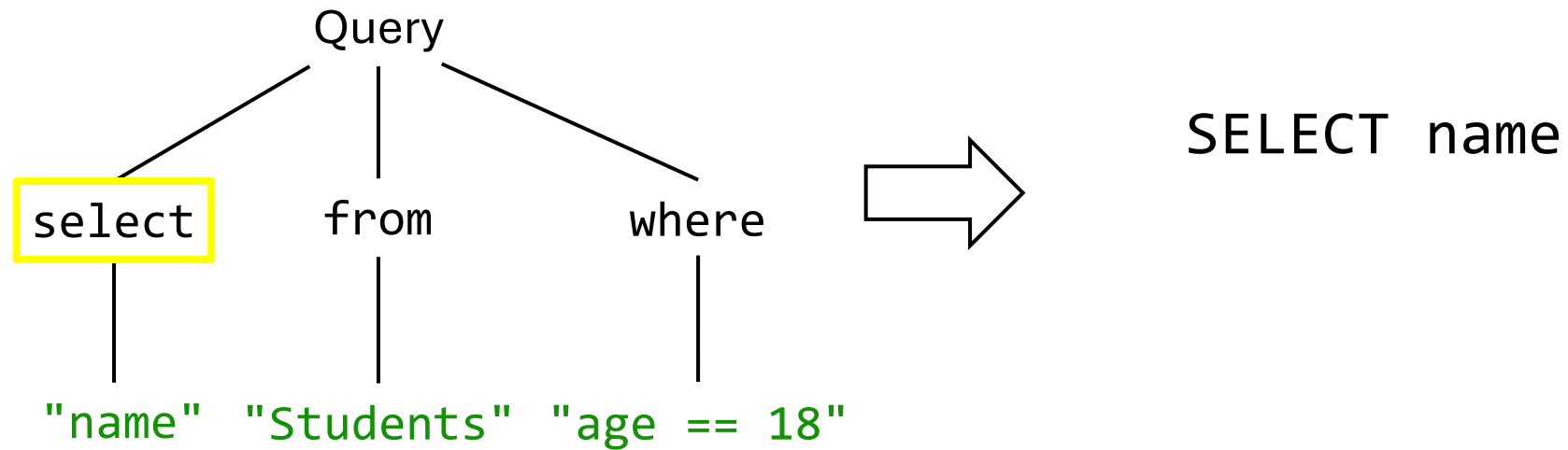
# Proposal

## 2. Backtranslation



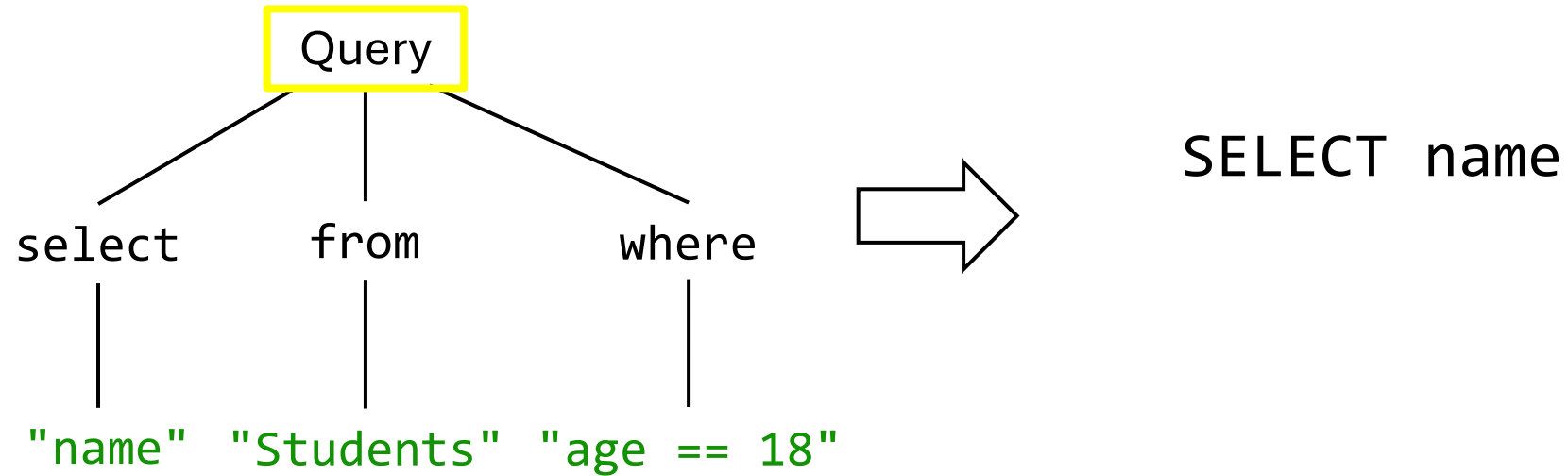
# Proposal

## 2. Backtranslation



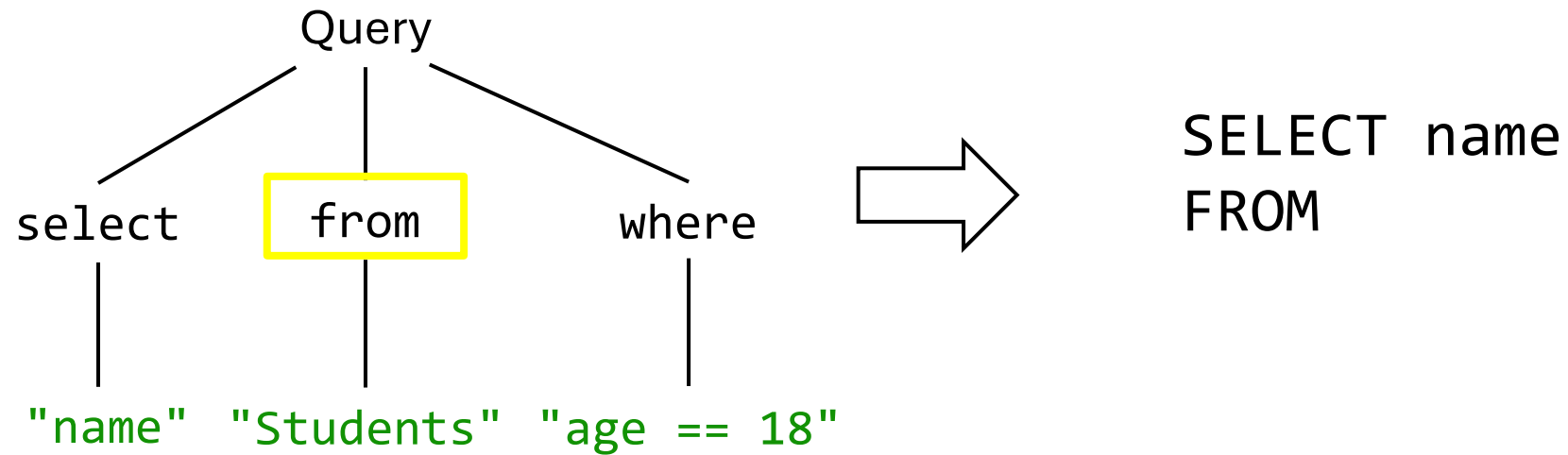
# Proposal

## 2. Backtranslation



# Proposal

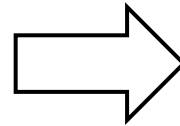
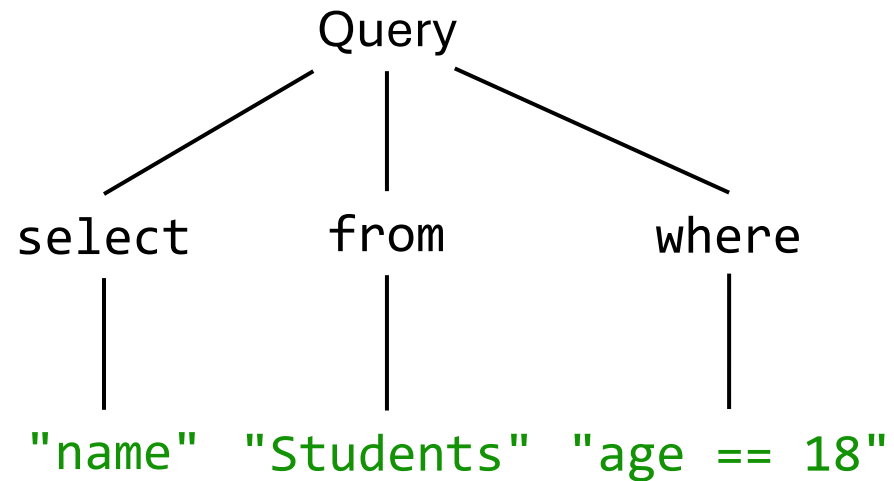
## 2. Backtranslation





# Proposal

## 2. Backtranslation

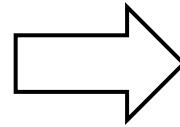


```
SELECT name  
FROM Students  
WHERE age == 18
```

# Proposal

## 3. Analysis

```
SELECT name  
FROM Students  
WHERE age == 18
```



```
SELECT name  
FROM Students  
WHERE age == 18
```

Parse Successful 

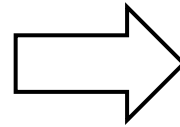
1 Semantic Error 

# Proposal

## 4. Projection

```
SELECT name  
FROM Students  
WHERE age == 18
```

Parse Successful   
1 Semantic Error 



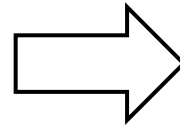
```
class Bar {  
    ...  
    public void foo() {  
        ...  
        Query query = SQL.select("name")  
                           .from("Students")  
                           .where("age == 18");  
        ...  
    }  
}
```

# Proposal

## 4. Projection

```
SELECT name
FROM Students
WHERE age == 18
```

Parse Successful   
1 Semantic Error 



```
class Bar {
    ...
    public void foo() {
        ...
        Query query = SQL.select("name")
                           .from("Students")
                           .where("age == 18");
        ...
    }
}
```

Unselected column 'age'  
[Select 'age' from 'Students'](#)

# Challenges & Limitations

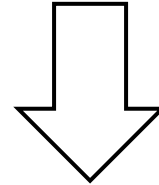
- Users will need to install an editor plug-in
- Dynamic parameters are problematic...

## Proposal

# Dynamic Parameters

EDSL Code

```
SQL.select(field).from("Students")
```



DSL Template

```
SELECT {field} FROM Students
```

Next step:

- Find value using dataflow analysis
- Otherwise, insert some valid value

# Current Progress



Come up with new method



Create a Prototype



Test it on existing Code

# Summary

- Fluent APIs for embedded DSLs are very useful!
- ...static analysis is very limited
- My proposed method:

1. Find  
embedded  
language code

2. Translate to  
original DSL

3. Analyze w/  
existing tools

4. Project  
results onto  
input